

INFORME DEL PROYECTO

GRUPO CORDILLERA

Plataforma de Monitoreo Ejecutivo Grupo Cordillera

Fecha: 18 de junio de 2026

Autor: Matías Medina

Índice

Contenido

Índice	2
Contextualización	3
2. Diagrama de microservicios	3
3. Descripción de la Persistencia	5
3.1 Stack Tecnológico de Persistencia.....	5
3.2 Patrón de Arquitectura de Persistencia	5
4. Implementación de Swagger para Endpoints	6
4.1 URLs de Swagger por Microservicio	6
5. Recurso de Persistencia (JPA)	10
5.1 Entidades Principales	10
5.1.1 Microservicio authentication - Entidad Usuario.....	10
5.1.2 Microservicio clientes - Entidad Cliente	10
5.1.3 Microservicio alertas - Entidad Alerta	10
5.2 Repositorios Spring Data JPA.....	11
5.2.1 UsuarioRepository	11
5.2.2 ClienteRepository	11
5.2.3 VentasRepository	12
5.2.4 ProductoRepository.....	12
5.2.5 MovimientoFinancieroRepository.....	13
5.2.6 AlertaRepository.....	14
5.3 Configuración de Base de Datos.....	14
6. Informe y Métricas de Pruebas Unitarias	15
6.1 Herramientas Utilizadas.....	15
6.2 Configuración de JaCoCo	15
6.3 Métricas de Cobertura (Ejemplo)	16
7. Descripción Pruebas Unitarias por Componente	17
7.1 Microservicio authentication - Pruebas de AuthService	17
7.2 Microservicio Inventario - Pruebas de InventarioService	18
7.3 Microservicio Ventas - Pruebas de VentasService	19
7.4 Microservicio Finanzas - Pruebas de FinanzasService	20

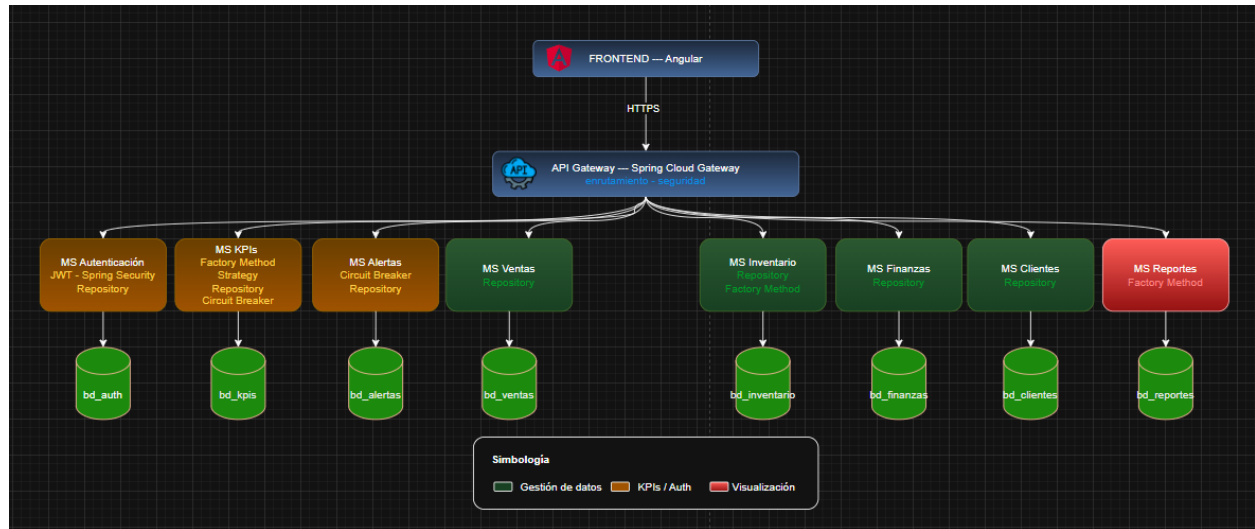
8. Conclusiones.....	20
----------------------	----

Contextualización

Grupo Cordillera es una empresa del sector retail y comercialización de productos para el hogar y tecnología, con presencia en distintas ciudades del país a través de diversas sucursales. En los últimos años, la organización ha experimentado un crecimiento constante de sus operaciones, ampliando su oferta de productos y extendiéndose hacia nuevos mercados. Este desarrollo ha traído consigo importantes desafíos en la gestión y análisis de información. Actualmente la empresa utiliza diversos sistemas independientes, como puntos de ventas (POS), plataformas de comercio electrónico, sistemas de inventario, herramientas financieras y soluciones de atención al cliente. La coexistencia de estas plataformas provocó una dispersión de los datos, dificultando obtener una visión integral del rendimiento organizacional. Como resultado, los equipos deben recurrir a procesos manuales para generar reportes, incluyendo la extracción de datos desde múltiples sistemas, su consolidación en hojas de cálculo, el formateo de registro de la información, la validación manual de indicadores y la preparación de informes para instancias ejecutivas. Este proceso genera demoras, aumenta el riesgo de errores, limita el acceso en tiempo real a información clave y crea una fuerte dependencia de tareas manuales. Para abordar esta situación, Grupo Cordillera ha decidido implementar una solución tecnológica basada en microservicios, orientada a integrar la información de sus distintos sistemas internos y a ofrecer herramientas de monitoreo en tiempo real de desempeño organizacional.

2. Diagrama de microservicios

La figura a continuación muestra la arquitectura de microservicios propuesta para la plataforma de Grupo Cordillera, el diagrama ilustra el flujo de comunicación desde el Frontend (angular) hacia la Api Gateway, el cual actúa como punto de enrutamiento, redirigiendo las solicitudes a cada microservicio según su dominio funcional. Cada microservicio maneja su propia base de datos (en caso de tenerla) independiente, garantizando de esta forma el aislamiento de datos y escalabilidad individual de cada componente. Los colores representan los módulos funcionales del sistema según su simbología.



3. Descripción de la Persistencia

La persistencia de datos en el proyecto Grupo Cordillera se implementa utilizando JPA con Hibernate como proveedor de ORM y Spring Data JPA para abstraer la capa de acceso a datos, la base de datos utilizada es PostgreSQL.

3.1 Stack Tecnológico de Persistencia

Tecnología	Versión/Descripción
JPA/Hibernate	Proveedor de persistencia incluido en Spring Boot Starter Data JPA
Spring Data JPA	Abstracción para repositorios CRUD y consultas derivadas (query methods)
PostgreSQL	Base de datos relacional
Lombok	Generación automática de getters, setters, constructores y más

3.2 Patrón de Arquitectura de Persistencia

Se sigue el patrón Repository-Service-Controller:

- Entidad (@Entity): Clase Java mapeada a una tabla de BD.
- Repositorio (JpaRepository): Interfaz que extiende JpaRepository para operaciones CRUD.
- Servicio: Capa de negocio que usa el repositorio.
- Controlador: Expone endpoints REST para interactuar con los datos.

4. Implementación de Swagger para Endpoints

La documentación de APIs REST se implementó con SpringDoc OpenAPI (generación automática de Swagger UI).

4.1 URLs de Swagger por Microservicio

Microservicio	URL de Swagger UI
Authentication	http://localhost:8081/swagger-ui/index.html
Ventas	http://localhost:8082/swagger-ui/index.html
Inventario	http://localhost:8083/swagger-ui/index.html
Finanzas	http://localhost:8084/swagger-ui/index.html
Clientes	http://localhost:8085/swagger-ui/index.html
Kpis	http://localhost:8086/swagger-ui/index.html
Alertas	http://localhost:8087/swagger-ui/index.html
Reportes	http://localhost:8088/swagger-ui/index.html
API Gateway	http://localhost:8080/swagger-ui/index.html

Autenticación Registro, inicio de sesión y administración de usuarios

PUT	/auth/admin/usuarios/{id}/rechazar	▼
PUT	/auth/admin/usuarios/{id}/desbloquear	▼
PUT	/auth/admin/usuarios/{id}/bloquear	▼
PUT	/auth/admin/usuarios/{id}/aprobar	▼
POST	/auth/registro	▼
POST	/auth/login	▼
GET	/auth/admin/usuarios	▼
GET	/auth/admin/roles	▼
GET	/auth/admin/pendientes	▼

Schemas

UsuarioAdminResponse > Expand all object

AprobarRequest > Expand all object

RegistroRequest > Expand all object

LoginRequest > Expand all object

LoginResponse > Expand all object

Ventas Registro, consulta y reproceso de ventas		^
GET	/ventas	▼
POST	/ventas	▼
POST	/ventas/finanzas/reprocesar	▼
GET	/ventas/total/sucursal/{sucursal}	▼
GET	/ventas/total/periodo	▼
GET	/ventas/sucursal/{sucursal}	▼
GET	/ventas/rango	▼
GET	/ventas/hoy	▼
GET	/ventas/finanzas/pendientes	▼
GET	/ventas/fecha/{fecha}	▼
DELETE	/ventas/{id}	▼
Schemas		^
Venta > Expand all object		

Inventario Registro, consulta y descuento de stock		^
PUT	/inventario/descontar/{sku}	▼
PUT	/inventario/descontar/{sku}/{cantidad}	▼
GET	/inventario	▼
POST	/inventario	▼
GET	/inventario/sucursal/{sucursal}	▼
GET	/inventario/sku/{sku}	▼
Schemas		^
DescuentoStockRequest > Expand all object		
Producto > Expand all object		
ProductoHogar > Expand all (any & object)		
ProductoTecnologia > Expand all (any & object)		

Finanzas Movimientos, totales y resúmenes financieros

GET	/finanzas	▼
POST	/finanzas	▼
GET	/finanzas/{id}	▼
GET	/finanzas/total	▼
GET	/finanzas/total/sucursal/{sucursal}	▼
GET	/finanzas/total/rango	▼
GET	/finanzas/tipo/{tipoMovimiento}	▼
GET	/finanzas/sucursal/{sucursal}	▼
GET	/finanzas/rango	▼
GET	/finanzas/fecha/{fecha}	▼

Schemas

MovimientoFinancieroRequest > Expand all object

MovimientoFinanciero > Expand all object

ResumenFinancieroDTO > Expand all object

Clientes Gestión de clientes y compras

GET	/clientes/{id}	▼
PUT	/clientes/{id}	▼
PUT	/clientes/{id}/compras	▼
GET	/clientes	▼
POST	/clientes	▼
GET	/clientes/tipo/{tipoCliente}	▼
GET	/clientes/estado/{estado}	▼

Schemas

ClienteRequest > Expand all object

Cliente > Expand all object

RegistrarCompraRequest > Expand all object

KPIs Resumen y detalle de indicadores ejecutivos

GET	/kpis	▼
GET	/kpis/{tipo}	▼

Schemas

KpiResult > Expand all object

Alertas Creación, consulta y detección de alertas

PUT	/alertas/{id}/leer	▼
PUT	/alertas/leer-todas	▼
GET	/alertas	▼
POST	/alertas	▼
POST	/alertas/detectar	▼
GET	/alertas/resumen	▼
GET	/alertas/no-leidas	▼
GET	/alertas/no-leidas/count	▼

Schemas

AlertaResponse > Expand all object

CrearAlertaRequest > Expand all object

AlertasResumenResponse > Expand all object

Reportes Generación de reportes PDF de ventas, inventario y finanzas

GET	/reportes/ventas/pdf	▼
GET	/reportes/inventario/pdf	▼
GET	/reportes/finanzas/pdf	▼

5. Recurso de Persistencia (JPA)

5.1 Entidades Principales

5.1.1 Microservicio authentication - Entidad Usuario

Archivo: *authentication/src/main/java/com/autenticacion/authentication/model/Usuario.java*

```
@Entity
@Table(name = "usuarios")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String username;

    @Column(nullable = false)
    private String password;

    private boolean cuentaBloqueada;
}
```

5.1.2 Microservicio clientes - Entidad Cliente

Archivo: *clientes/src/main/java/com/grupocordillera/clientes/model/Cliente.java*

```
@Entity
@Table(name = "clientes")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Cliente {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String nombre;

    @Column(nullable = false, unique = true)
    private String rut;

    private String email;
}
```

5.1.3 Microservicio alertas - Entidad Alerta

Archivo: *alertas/src/main/java/com/grupocordillera/alertas/model/Alerta.java*

```
@Entity
@Table(name = "alertas")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Alerta {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String descripcion;

    private String nivel;

    private LocalDateTime fechaCreacion;
}
```

5.2 Repositorios Spring Data JPA

5.2.1 UsuarioRepository

Archivo:

authentication/src/main/java/com/autenticacion/authentication/repository/UsuarioRepository.java

```
@Repository
public interface UsuarioRepository extends JpaRepository<Usuario, Long> {

    Optional<Usuario> findByEmail(String email);

    Optional<Usuario> findByUsername(String username);

    List<Usuario> findByEstado(Usuario.Estado estado);
}
```

5.2.2 ClienteRepository

Archivo:

clientes/src/main/java/com/grupocordillera/clientes/repository/ClienteRepository.java

```
@Repository
public interface ClienteRepository extends JpaRepository<Cliente, Long> {

    Optional<Cliente> findByEmailIgnoreCase(String email);

    boolean existsByEmailIgnoreCase(String email);

    List<Cliente> findByTipoCliente(Cliente.TipoCliente tipoCliente);

    List<Cliente> findByEstado(Cliente.Estado estado);
}
```

5.2.3 VentasRepository

Archivo: *ventas/src/main/java/com/grupocordillera/ventas/repository/VentasRepository.java*

```
@Repository
public interface VentasRepository extends JpaRepository<Venta, Long> {

    List<Venta> findByFecha(LocalDate fecha);

    List<Venta> findBySucursal(String sucursal);

    List<Venta> findByFechaBetween(LocalDate inicio, LocalDate fin);

    List<Venta> findByCanal(Venta.Canal canal);

    List<Venta> findByEstadoFinanzasIn(Collection<Venta.EstadoFinanzas> estados);

    @Query("SELECT SUM(v.montoTotal) FROM Venta v WHERE v.fecha BETWEEN :inicio AND :fin")
    Double sumMontoTotalByFechaBetween(LocalDate inicio, LocalDate fin);

    @Query("SELECT SUM(v.montoTotal) FROM Venta v WHERE v.sucursal = :sucursal")
    Double sumMontoTotalBySucursal(String sucursal);

}
```

5.2.4 ProductoRepository

Archivo:

inventario/src/main/java/com/grupocordillera/inventario/repository/ProductoRepository.java

```
@Repository
public interface ProductoRepository extends JpaRepository<Producto, Long> {

    Optional<Producto> findBySku(String sku);

    List<Producto> findBySkuStartingWith(String skuPrefix);

    List<Producto> findBySucursal(String sucursal);

    List<Producto> findByCategoria(String categoria);

    List<Producto> findByEstado(Producto.Estado estado);

}
```

5.2.5 MovimientoFinancieroRepository

Archivo:

finanzas/src/main/java/com/grupocordillera/finanzas/repository/MovimientoFinancieroRepository.java

@Repository

```
public interface MovimientoFinancieroRepository extends
JpaRepository<MovimientoFinanciero, Long> {

    Optional<MovimientoFinanciero> findByVentaId(Long ventaId);

    List<MovimientoFinanciero> findByFechaRegistro(LocalDate fechaRegistro);

    List<MovimientoFinanciero> findBySucursalIgnoreCase(String sucursal);

    List<MovimientoFinanciero> findByTipoMovimiento(MovimientoFinanciero.TipoMovimiento
tipoMovimiento);

    List<MovimientoFinanciero> findByFechaRegistroBetween(LocalDate inicio, LocalDate
fin);

    @Query("SELECT COALESCE(SUM(m.ingresos), 0) FROM MovimientoFinanciero m")
    BigDecimal sumIngresos();

    @Query("SELECT COALESCE(SUM(m.costos), 0) FROM MovimientoFinanciero m")
    BigDecimal sumCostos();

    @Query("SELECT COALESCE(SUM(m.margen), 0) FROM MovimientoFinanciero m")
    BigDecimal sumMargen();

    @Query("SELECT COALESCE(SUM(m.ingresos), 0) FROM MovimientoFinanciero m WHERE
m.fechaRegistro BETWEEN :inicio AND :fin")
    BigDecimal sumIngresosByFechaRegistroBetween(LocalDate inicio, LocalDate fin);

    @Query("SELECT COALESCE(SUM(m.costos), 0) FROM MovimientoFinanciero m WHERE
m.fechaRegistro BETWEEN :inicio AND :fin")
    BigDecimal sumCostosByFechaRegistroBetween(LocalDate inicio, LocalDate fin);

    @Query("SELECT COALESCE(SUM(m.margen), 0) FROM MovimientoFinanciero m WHERE
m.fechaRegistro BETWEEN :inicio AND :fin")
    BigDecimal sumMargenByFechaRegistroBetween(LocalDate inicio, LocalDate fin);
```

```

    @Query("SELECT COALESCE(SUM(m.ingresos), 0) FROM MovimientoFinanciero m WHERE LOWER(m.sucursal) = LOWER(:sucursal)")

    BigDecimal sumIngresosBySucursal(String sucursal);

    @Query("SELECT COALESCE(SUM(m.costos), 0) FROM MovimientoFinanciero m WHERE LOWER(m.sucursal) = LOWER(:sucursal)")

    BigDecimal sumCostosBySucursal(String sucursal);

    @Query("SELECT COALESCE(SUM(m.margen), 0) FROM MovimientoFinanciero m WHERE LOWER(m.sucursal) = LOWER(:sucursal)")

    BigDecimal sumMargenBySucursal(String sucursal);
}

```

5.2.6 AlertaRepository

Archivo:

alertas/src/main/java/com/grupocordillera/alertas/repository/AlertaRepository.java

```

@Repository

public interface AlertaRepository extends JpaRepository<Alerta, Long> {

    Optional<Alerta> findByCodigo(String codigo);

    long countByLeidaFalse();

    List<Alerta> findByLeidaFalseOrderByFechaCreacionDesc();

    List<Alerta> findAllByOrderByFechaCreacionDesc();
}

```

5.3 Configuración de Base de Datos

Archivo: *authentication/src/main/resources/application.properties*

```

spring.datasource.url=jdbc:postgresql://localhost:5432/auth_db
spring.datasource.username=postgres
spring.datasource.password=postgres
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

```

6. Informe y Métricas de Pruebas Unitarias

Las pruebas unitarias del backend se desarrollaron utilizando JUnit 5 en conjunto con Mockito, que corresponden al stack de testing estándar y recomendado oficialmente para aplicaciones Spring Boot, ya que se integran de forma nativa a través de la dependencia spring-boot-starter-test.

6.1 Herramientas Utilizadas

Categoría	Herramienta
Testing Backend	JUnit 5 + Mockito
Cobertura	JaCoCo

6.2 Configuración de JaCoCo

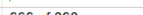
El plugin de JaCoCo está configurado en el pom.xml de cada microservicio para generar reportes de cobertura.

Extracto de pom.xml:

```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.11</version>
  <executions>
    <execution>
      <goals>
        <goal>prepare-agent</goal>
      </goals>
    </execution>
    <execution>
      <id>report</id>
      <phase>test</phase>
      <goals>
        <goal>report</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```

6.3 Métricas de Cobertura

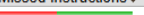
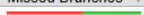
authentication

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
com.autenticacion.authentication.service		25 %		23 %	39 43	80 104	23 25	1 2
com.autenticacion.authentication.config		0 %		0 %	15 15	45 45	8 8	4 4
com.autenticacion.authentication.controller		0 %		n/a	12 12	15 15	12 12	1 1
com.autenticacion.authentication		0 %		n/a	2 2	3 3	2 2	1 1
com.autenticacion.authentication.model		100 %		n/a	0 2	0 4	0 2	0 2
com.autenticacion.authentication.exception		100 %		n/a	0 1	0 2	0 1	0 1
Total	666 of 868	23 %	40 of 48	16 %	68 75	143 173	45 50	7 11

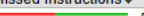
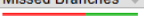
inventario

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
com.grupocordillera.inventario.service		57 %		37 %	15 24	17 46	4 12	0 1
com.grupocordillera.inventario.config		0 %		n/a	3 3	13 13	3 3	1 1
com.grupocordillera.inventario.controller		0 %		n/a	6 6	6 6	6 6	1 1
com.grupocordillera.inventario.factory		0 %		0 %	5 5	7 7	2 2	1 1
com.grupocordillera.inventario.model		90 %		62 %	3 9	2 19	0 5	0 2
com.grupocordillera.inventario		0 %		n/a	2 2	3 3	2 2	1 1
Total	254 of 484	47 %	23 of 37	37 %	34 49	48 94	17 30	4 7

finanzas

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
com.grupocordillera.finanzas.service		56 %		44 %	26 34	24 61	10 17	0 1
com.grupocordillera.finanzas.config		0 %		n/a	5 5	13 13	5 5	1 1
com.grupocordillera.finanzas.controller		0 %		n/a	10 10	10 10	10 10	1 1
com.grupocordillera.finanzas.model		90 %		62 %	3 9	2 21	0 5	0 2
com.grupocordillera.finanzas		0 %		n/a	2 2	3 3	2 2	1 1
Total	280 of 562	50 %	22 of 42	47 %	46 60	52 108	27 39	3 6

ventas

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
com.grupocordillera.ventas.service		53 %		38 %	46 61	67 146	15 26	1 2
com.grupocordillera.ventas.config		0 %		n/a	3 3	13 13	3 3	1 1
com.grupocordillera.ventas.controller		0 %		n/a	11 11	13 13	11 11	1 1
com.grupocordillera.ventas.dto		0 %		n/a	1 1	4 4	1 1	1 1
com.grupocordillera.ventas.model		93 %		62 %	3 11	2 30	0 7	0 4
com.grupocordillera.ventas		0 %		n/a	2 2	3 3	2 2	1 1
Total	460 of 939	51 %	46 of 78	41 %	66 89	102 209	32 50	5 10

7. Descripción Pruebas Unitarias por Componente

Todas las pruebas unitarias del backend están implementadas con JUnit 5 (anotaciones `@Test`, `@BeforeEach`, `@ExtendWith`) y Mockito (`@Mock`, `@InjectMocks`, `Mockito.when`) para el aislamiento de dependencias, siguiendo las buenas prácticas recomendadas por Spring Boot para testing de la capa de servicio.

7.1 Microservicio authentication - Pruebas de AuthService

```
[INFO] -----< com.autenticacion:authentication >-----
[INFO] Building authentication 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- clean:3.5.0:clean (default-clean) @ authentication ---
[INFO] Deleting C:\Users\sapbe\Documents\Grupo Cordillera\grupo-cordillera-backend\authentication\target
[INFO]
[INFO] --- jacoco:0.8.12:prepare-agent (prepare-agent) @ authentication ---
[INFO] argLine set to "-javaagent:C:\Users\sapbe\.m2\repository\org\jacoco\org.jacoco.agent\0.8.12\org.jacoco.agent-0.8.12-runtime.jar=dest
file=C:\Users\sapbe\Documents\Grupo Cordillera\grupo-cordillera-backend\authentication\target\jacoco.exec"
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ authentication ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.14.1:compile (default-compile) @ authentication ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 16 source files with javac [debug parameters release 17] to target/classes
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ authentication ---
[INFO] skip non existing resourceDirectory C:\Users\sapbe\Documents\Grupo Cordillera\grupo-cordillera-backend\authentication\src\test\resources
[INFO]
[INFO] --- compiler:3.14.1:testCompile (default-testCompile) @ authentication ---
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 2 source files with javac [debug parameters release 17] to target\test-classes
[INFO]
[INFO] --- surefire:3.5.5:test (default-test) @ authentication ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.autenticacion.authentication.AuthenticationApplicationTests
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.216 s -- in com.autenticacion.authentication.AuthenticationApplicationTests
[INFO] Running com.autenticacion.authentication.service.AuthServiceTest
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as a
n agent to your build as described in Mockito's documentation: https://javadoc.io/doc/org.mockito/mockito-core/latest/org.mockito/org/mockito/Mockit
o.html#0.3
WARNING: A Java agent has been loaded dynamically (C:\Users\sapbe\.m2\repository\net\bytebuddy\byte-buddy-agent\1.17.8\byte-buddy-agent-1.17.8.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.461 s -- in com.autenticacion.authentication.service.AuthServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 12.661 s
[INFO] Finished at: 2026-06-18T21:38:24-04:00
[INFO] -----
C:\Users\sapbe\Documents\Grupo Cordillera\grupo-cordillera-backend\authentication>mvn jacoco:report
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.autenticacion:authentication >-----
```

7.2 Microservicio Inventario - Pruebas de InventarioService

```
[INFO] -----< com.grupocordillera:inventario >-----
[INFO] Building inventario 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] --- clean:3.5.0:clean (default-clean) @ inventario ---
[INFO] Deleting C:\Users\sapbe\Documents\Grupo Cordillera\grupo-cordillera-backend\inventario\target
[INFO] --- jacoco:0.8.12:prepare-agent (prepare-agent) @ inventario ---
[INFO] argLine set to "-javaagent:C:\Users\sapbe\.m2\repository\org\jacoco\org.jacoco.agent\0.8.12\org.jacoco.agent-0.8.12-runtime.jar=dest
file=C:\Users\sapbe\Documents\Grupo Cordillera\grupo-cordillera-backend\inventario\target\jacoco.exec"
[INFO] --- resources:3.3.1:resources (default-resources) @ inventario ---
[INFO] Copying 1 resource from src\main\resources to target\classes
[INFO] Copying 0 resource from src\main\resources to target\classes
[INFO] --- compiler:3.14.1:compile (default-compile) @ inventario ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 11 source files with javac [debug parameters release 17] to target\classes
[INFO] --- resources:3.3.1:testResources (default-testResources) @ inventario ---
[INFO] skip non existing resourceDirectory C:\Users\sapbe\Documents\Grupo Cordillera\grupo-cordillera-backend\inventario\src\test\resources
[INFO] --- compiler:3.14.1:testCompile (default-testCompile) @ inventario ---
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 2 source files with javac [debug parameters release 17] to target\test-classes
[INFO] --- surefire:3.5.5:test (default-test) @ inventario ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.grupocordillera.inventario.InventarioApplicationTests
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.086 s -- in com.grupocordillera.inventario.InventarioApplicationTests
[INFO] Running com.grupocordillera.inventario.service.InventarioServiceTest
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as a
n agent to your build as described in Mockito's documentation: https://javadoc.io/doc/org.mockito/mockito-core/latest/org.mockito/org/mockito/Mockit
o.html#0.3
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
WARNING: A Java agent has been loaded dynamically (C:\Users\sapbe\.m2\repository\net\bytebuddy\byte-buddy-agent\1.17.8\byte-buddy-agent-1.17.8.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.807 s -- in com.grupocordillera.inventario.service.InventarioServiceTest
[INFO] Results:
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 9.596 s
[INFO] Finished at: 2026-06-18T21:42:50-04:00
[INFO] -----
```

7.3 Microservicio Ventas - Pruebas de VentasService

```
[INFO] -----< com.grupocordillera.ventas >-----
[INFO] Building ventas 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- clean:3.5.0:clean (default-clean) @ ventas ---
[INFO] Deleting C:\Users\sapbe\Documents\Grupo Cordillera\grupo-cordillera-backend\ventas\target
[INFO]
[INFO] --- jacoco:0.8.12:prepare-agent (prepare-agent) @ ventas ---
[INFO] argLine set to "-javaagent:C:\Users\sapbe\.m2\repository\org\jacoco\org.jacoco.agent\0.8.12\org.jacoco.agent-0.8.12-runtime.jar-dest
file=C:\Users\sapbe\Documents\Grupo Cordillera\grupo-cordillera-backend\ventas\target\jacoco.exec"
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ ventas ---
[INFO] Copying 1 resource from src\main\resources to target\classes
[INFO] Copying 0 resource from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.14.1:compile (default-compile) @ ventas ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 16 source files with javac [debug parameters release 17] to target\classes
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ ventas ---
[INFO] skip non existing resourceDirectory C:\Users\sapbe\Documents\Grupo Cordillera\grupo-cordillera-backend\ventas\src\test\resources
[INFO]
[INFO] --- compiler:3.14.1:testCompile (default-testCompile) @ ventas ---
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 2 source files with javac [debug parameters release 17] to target\test-classes
[INFO]
[INFO] --- surefire:3.5.5:test (default-test) @ ventas ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.grupocordillera.ventas.service.VentasServiceTest
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as a
n agent to your build as described in Mockito's documentation: https://javadoc.io/doc/org.mockito/mockito-core/latest/org.mockito/org/mockito/Mockit
o.html#0.3
WARNING: A Java agent has been loaded dynamically (C:\Users\sapbe\.m2\repository\net\bytebuddy\byte-buddy-agent\1.17.8\byte-buddy-agent-1.17.8.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.424 s -- in com.grupocordillera.ventas.service.VentasServiceTest
[INFO] Running com.grupocordillera.ventas.VentasApplicationTests
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.002 s -- in com.grupocordillera.ventas.VentasApplicationTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 10.541 s
[INFO] Finished at: 2026-06-18T21:47:14-04:00
[INFO] -----
```

7.4 Microservicio Finanzas - Pruebas de FinanzasService

```
[INFO] -----< com.grupocordillera:finanzas >-----
[INFO] Building finanzas 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- clean:3.5.0:clean (default-clean) @ finanzas ---
[INFO] Deleting C:\Users\sapbe\Documents\Grupo Cordillera\grupo-cordillera-backend\finanzas\target
[INFO]
[INFO] --- jacoco:0.8.12:prepare-agent (prepare-agent) @ finanzas ---
[INFO] argLine set to "-javaagent:C:\Users\sapbe\m2\repository\org\jacoco\org.jacoco.agent\0.8.12\org.jacoco.agent-0.8.12-runtime.jar=dest
File=C:\Users\sapbe\Documents\Grupo Cordillera\grupo-cordillera-backend\finanzas\target\jacoco.exec"
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ finanzas ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.14.1:compile (default-compile) @ finanzas ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 11 source files with javac [debug parameters release 17] to target/classes
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ finanzas ---
[INFO] skip non existing resourceDirectory C:\Users\sapbe\Documents\Grupo Cordillera\grupo-cordillera-backend\finanzas\src\test\resources
[INFO]
[INFO] --- compiler:3.14.1:testCompile (default-testCompile) @ finanzas ---
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 2 source files with javac [debug parameters release 17] to target\test-classes
[INFO]
[INFO] --- surefire:3.5.5:test (default-test) @ finanzas ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.grupocordillera.finanzas.FinanzasApplicationTests
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.089 s -- in com.grupocordillera.finanzas.FinanzasApplicationTests
[INFO] Running com.grupocordillera.finanzas.service.FinanzasServiceTest
[INFO] Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as a
n agent to your build as described in Mockito's documentation: https://javadoc.io/doc/org.mockito/mockito-core/latest/org.mockito/org/mockito/Mockit
o.html#0.3
[INFO] WARNING: A Java agent has been loaded dynamically (C:\Users\sapbe\m2\repository\net\bytebuddy\byte-buddy-agent\1.17.8\byte-buddy-agent-1.17.8.jar)
[INFO] WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
[INFO] WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
[INFO] WARNING: Dynamic loading of agents will be disallowed by default in a future release
[INFO] Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.001 s -- in com.grupocordillera.finanzas.service.FinanzasServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 10.440 s
[INFO] Finished at: 2026-06-18T21:46:01-04:00
[INFO] -----
```

8. Conclusiones

La plataforma Grupo Cordillera se desarrolló bajo una arquitectura de microservicios, con un API Gateway como punto de entrada único para centralizar el acceso y la seguridad, la cual se implementa mediante tokens JWT para la autenticación y autorización de usuarios.

Para la capa de persistencia, se adoptó JPA con Hibernate como proveedor y Spring Data JPA como abstracción para simplificar las operaciones CRUD y consultas derivadas, usando PostgreSQL como base de datos relacional, además se utilizaron patrones de diseño como el Factory Method para la creación de productos y calculadoras de KPIs, y el Strategy Pattern para la lógica variable de los KPIs, y en el microservicio de KPIs se agregó un Circuit Breaker para mejorar la resiliencia ante fallos en otros servicios. Para garantizar la calidad del código, se desarrollaron pruebas unitarias con JUnit 5 y Mockito, enfocadas en los servicios de

authentication, ventas, inventario, finanzas y clientes, con cobertura medida mediante JaCoCo, y por último, para documentar los endpoints REST de forma clara y accesible, se integró Swagger UI con SpringDoc OpenAPI tanto en el API Gateway como en cada uno de los microservicios individuales.